Script Composer Evolution — v1.0 Architecture

> **Status:** Adopted · **Version:** 1.0 · **Owner:** Prasanth (LevelUp AI QA)
> **Applies to:** `levelup-ai-qa-agent` (backend script generation) and its dashboard consumer.
> **This document is the reference for every Script Composer PR going forward. Cite it in each PR.**

---

0. Purpose

The scenario pipeline is done. Phases A/B (Scenario Data Model + Renderers — PR #247) gave us a single canonical scenario, projected per consumer. We are **not** going to keep redesigning the pipeline.

From here, we make the **existing Script Composer progressively smarter** through deterministic engineering heuristics, ranking algorithms, and quality refinements — never through new orchestrators, intelligence providers, writers, planners, or pipelines.

This document freezes the architecture at **v1.0** and defines the incremental roadmap (Sprints 1, 1.5, 2–7 + the two deterministic rule libraries) that raises generated-code quality without destabilizing the foundation.

---

1. The Freeze Mandate (read this first)

> **Freeze the architecture. Do not introduce new orchestrators, intelligence providers, writers,
> planners, or pipelines. All future improvements must occur inside the existing Script Composer
> through deterministic engineering heuristics, ranking algorithms, and quality refinements. Every
> PR must make the generated framework look more like code written by an experienced automation
> engineer while preserving existing capabilities and avoiding regressions.**

The single measure of success

Every future PR is judged by one question:

> **“Would a senior Playwright engineer approve this pull request without realizing it was AI-generated?”**

If a change does not move us toward **yes**, it does not belong in this roadmap.

What this explicitly forbids

- ❌ New orchestrators / intelligence providers / writers / planners / pipelines
- ❌ Moving responsibilities between frozen components (see §2)

- ❌ More comments, AI explaining every action, verbose logging
- ❌ Huge helper methods, generated documentation, over-engineering
- ❌ Defaulting to the LLM when a deterministic heuristic can decide

What this encourages

- ✅ Better **decisions** inside existing engines (ranking, scoring, fallback)
- ✅ Deterministic heuristics over prompts (no token growth)
- ✅ Reuse of existing framework assets over regeneration
- ✅ Honest confidence signals over pretended certainty

---

2. The Frozen Pipeline (v1.0)

These are foundation components. **Freeze them. Do not keep moving responsibilities around.**

```
Requirement
    ↓
Scenario Builder          (engines/scenario-builder.ts — canonical scenario, Phase A)
    ↓
Scenario Integrity Validator (certifies the scenario; warnings only, never mutates — S
print 1.5)
    ↓
Canonical Scenario        (grounding[] + structured expected, one source of truth)
    ↓
Execution Context         (crawl + repo intelligence + app profile + conventions)
    ↓
Script Composer           (script-gen/script-gen-engine.ts — ScriptGenEngine)
    ├── Candidate Resolution   (reuse-first: fixture / page object / helper / compon-
ent / locator — Sprint 2)
    ├── Assertion Expansion    (Sprint 3)
    ├── Coverage Expansion     (Sprint 4)
    ├── Reuse Decisions        (Existing Code First heuristic — §5.1)
    └── Engineering Heuristics (deterministic rule library — §5.1)
    ↓
Quality Gate              (aggregates/presents quality dimensions — validation-runner
+ ai-review-engine + framework-auditor)
    ↓
Persist                   (generated_test_cases / scripts + ai_metadata)
```

> **This is the final architecture — it is frozen here.** Notice there is **not a single new pipeline stage** beyond the Validator. Everything the roadmap adds (Sprints 2–6) is behavior inside the existing Script Composer, shown as the sub-bullets above. No new orchestrators, no new validators, no moved stage boundaries.
>
> The **Scenario Integrity Validator** is a deterministic certifier inserted between the Scenario Builder and everything downstream. It does **not** own or mutate scenario data — the Scenario Builder remains the sole owner. It only answers: “Is this canonical scenario internally consistent and automation-ready?” and attaches a readiness score + warnings. See Sprint 1.5.

Where each frozen stage lives in the codebase

Stage	Primary module(s)
Scenario Builder	`src/engines/scenario-builder.ts`
Scenario Integrity Validator	`src/engines/scenario-integrity/` (proposed — Sprint 1.5)
Canonical Scenario	`DraftTestCase` / `FormatterTestCase` (`grounding[]` , structured `expected` , `schemaVersion`)
Renderers / Projection	`src/renderers/scenario-renderer.ts` (Manual / Script / BDD)
Execution Context	`src/script-gen/page-crawler.ts` , `src/context/*` , `src/intelligence/project-convention-profile.ts`
Script Composer	`src/script-gen/script-gen-engine.ts` (`ScriptGenEngine`)
Quality Gate	`src/script-gen/validation-runner.ts` , `ai-review-engine.ts` , `framework-auditor.ts`
Persist	`src/api/routes/script-gen.ts` , `test-coverage.ts`

Everything in the roadmap below happens **inside the Script Composer** and its owned sub-engines. No stage boundary moves.

---

3. Evolve the Script Composer — not the pipeline

The Script Composer (`ScriptGenEngine`) already composes the sub-engines we will sharpen. We are **not** adding engines; we are making the existing ones decide better.

Sub-engine (exists today)	Role	Sprint that sharpens it
`SelectorQualityEngine` + `intelligence/element-intel-li-gence.rankLocatorCandidates`	Locator scoring	**Sprint 2**
`AssertionEngine`	Assertions	**Sprints 3 & 4**
`project-convention-profile` (`buildReuseCatalogue` , `find-ReusablePageObject`)	Reuse (candidate resolution)	**Sprint 2**
`ai-review-engine` / `frame-work-auditor`	Style / audit	**Sprint 5**
`engines/confidence-en-gine.ts`	Confidence	**Sprint 6**
`core/candidate-ranker.ts` + `core/advisors/*`	Ranking (healing side)	**Reuse target for Sprint 2**

> **Reuse-first note:** the healing side already has a mature scored candidate ranker (`core/candidate-ranker.ts` with `SOURCE_TRUST` , plus `core/advisors/{rule-engine,app-profile,dom-candidate,dom-memory,learned-pattern,ai}-advisor.ts`). Sprint 2 should **reuse and extend** this ranking substrate for generation, not build a parallel one. One ranking brain, two callers.

---

4. Roadmap

Sprint 1 — Fix & Freeze ✅ (this PR set)

- ✅ Fix the **Generate Script** regression (Test Case Lab → Script Gen deep link now selects the requirement, prefills the scenario, and hides/omits App Knowledge because the test case is ready). (dashboard PR #146)
- ✅ Merge PR #247 (Scenario Data Model + Renderers). (owner action)
- ✅ Create this document (`SCRIPT_COMPOSER_EVOLUTION.md`) and **declare the architecture frozen.**
- **No new intelligence.**

Sprint 1.5 — Scenario Integrity Validator ⭐⭐⭐⭐⭐ (do before Sprint 2)

A deterministic **certifier** that sits between the Scenario Builder and the Script Composer. It is **not new intelligence** — no LLM, no generation. It answers one question:

> **“Is this canonical scenario internally consistent and automation-ready?”**

Today the Script Composer sometimes compensates for weak test cases. It shouldn't have to. If the Composer always receives a high-quality, certified scenario, locator / assertion / coverage mistakes drop **without touching generation.** This is the first of the two deterministic rule libraries (see §5) and the natural partner to the Engineering Heuristics Library.

> **It is a Validator, not an Engine.** The name is deliberate. Alongside Scenario **Builder**, Script **Composer**, and Quality **Gate**, calling this an "Engine" would imply it owns logic. It does not. It **certifies.** Its entire responsibility is: "Is this scenario internally consistent?" — nothing more.

Hard rule #1: report, never rewrite

```
Scenario → Validator → Issues → (same) Scenario     ✅
Scenario → Validator → Scenario rewritten           ❌  (forbidden)
```

The Scenario Builder remains the **sole owner** of scenario data. The Integrity Validator only attaches quality signals (warnings + a readiness score); it never mutates steps, grounding, or expected.

Hard rule #2: the readiness score influences confidence, NEVER behavior

The Automation Readiness Score must **never block generation.** Enterprise teams often intentionally automate incomplete requirements — the tool must not become restrictive.

```
Scenario Ready — 63%   →   Generation blocked.            ❌  (forbidden)

Scenario Ready — 63%   →   Generation completed.          ✅
                           Confidence: Medium
                           Warnings:
                             - Persona mismatch
                             - Missing grounding
```

The score influences **confidence and warnings only.** `generationAllowed` is always `true`.

Hard rule #3: it must NOT become a mini Script Composer

The Validator does **not** improve locators, expand assertions, add coverage, or touch the framework. Those belong to the Script Composer (Sprints 2–6). Keep the Validator brutally simple — its only job is internal-consistency certification.

Hard rule #4: the numeric score is INTERNAL — expose the band, not the number

The `0–100` readiness score is a useful **internal** signal (weighting, telemetry, trend analysis).
It must **never** reach a user-facing surface as a raw number. Externally — UI, API responses, PR comments, Smart TODOs — expose only the confidence band: **High** / **Medium** / **Low**.

```
Internal (ai_metadata):   readinessScore: 91        ← kept, for tuning & telemetry
External (UI / API):      Confidence: High           ← the only thing users see
```

Reason: a raw number implies false precision. Users cannot tell whether `91` is meaningfully better than `88` — but High vs. Medium is honest and actionable. The report keeps both fields (`readinessScore` internal, `confidence` external); every serializer that renders to a human surfaces `confidence` and hides `readinessScore`.

What it validates

> Every check below produces **warnings only** — never a hard, generation-blocking failure (Hard rule #2). “warn” means the check found an inconsistency and lowered the readiness score.

1. **Persona consistency** ⭐⭐⭐⭐⭐ — the persona/test data must match the scenario intent.
   - `Title: Login with locked user` + `Test Data: standard_user` → **warn**
   - `Title: Successful login` + `Expected: Authentication rejected` → **warn**
2. **Coverage polarity** ⭐⭐⭐⭐⭐ — expected outcome must match the coverage type (deterministic):
   - positive → success state (e.g. dashboard displayed)
   - negative → error shown
   - edge → graceful validation
   - boundary → limit accepted/rejected
3. **Test-data suitability** ⭐⭐⭐⭐⭐ — dataset must fit the scenario.
   - `valid login` scenario + `locked_user` dataset → **warn**
4. **Expected-result consistency** ⭐⭐⭐⭐⭐ — manual outcome vs. structured expected must agree.
   - manual `Login successful` + expected `Error displayed` → **warn**
5. **Step completeness** ⭐⭐⭐⭐⭐ — steps must plausibly produce the expected outcome.
   - `Click Login` → expected `Inventory page`, but no username/password entered → **warn (incomplete)**
6. **Missing preconditions** ⭐⭐⭐⭐⭐ — required preconditions must be present.
   - `Checkout` without `User logged in` → **warn**
7. **Business flow consistency** ⭐⭐⭐⭐⭐ — steps must follow a possible state progression.
   Deterministic state-order validation (no AI): each step maps to a business stage (`open → login → browse → add-to-cart → cart → checkout → payment → confirmation → logout`), and impossible orderings are flagged.
   - `Checkout → Payment → Add item` → **warn** (add-to-cart after checkout is impossible)
   - `Logout → Add item` → **warn** (authenticated action after logout is impossible)
   - `Checkout` with no prior `Add item` → **warn** (checkout without a cart)
8. **Grounding completeness** ⭐⭐⭐⭐☆ — every actionable step should have grounding.
   - `Enter username` with no grounding → **do not warn hard; reduce confidence** (score penalty only).

Automation Readiness Score

Before the Script Composer starts, the Validator computes a scenario-quality score so the Composer knows
how much to trust the scenario (confidence only — it never blocks, per Hard rule #2):

Check	Result
Persona	✅
Coverage polarity	✅
Test Data	✅
Expected	✅
Step completeness	✅
Preconditions	✅
Business flow	✅
Grounding	⚠️

```
Internal:   readinessScore 96   →   External:   Confidence: High
```

The internal score (and the per-check breakdown) is carried alongside the scenario and later feeds the Sprint 6 Confidence Engine and Smart TODOs — honest signal instead of silent compensation. Only the
band is ever shown to a user (Hard rule #4).

One number is not enough — but do NOT create more validators

A single readiness number isn't actionable, so the Quality Gate presents a multi-dimensional report. **Critical:** the extra dimensions are **not** new validators. There is exactly one validator — the Scenario Integrity Validator — and it owns only the **Scenario** dimension. Every other dimension is computed by a **sub-engine that already exists inside the Script Composer** and is simply surfaced
by the Quality Gate. We never add a Locator Validator, Assertion Validator, or Framework Validator; doing so would slowly recreate the architecture we just froze.

```
Scenario     High     (Scenario Integrity Validator — the ONE validator, this sprint)
Framework    High     (framework-auditor — existing Composer sub-engine)
Locator      High     (Candidate Resolution — Sprint 2, inside the Composer)
Assertion    High     (AssertionEngine — Sprints 3 & 4, inside the Composer)
─────────────────────
Overall      High
```

> Bands, not numbers, at every level (Hard rule #4). The Quality Gate is an **aggregator/presenter**, not a new validation layer — each dimension plugs in from an existing Composer sub-engine without a
> new category and without changing the others.

Where it lives

`src/engines/scenario-integrity/` — a pure, dependency-light rule module. Output is a typed `ScenarioIntegrityReport { readinessScore, confidence, generationAllowed, checks[], warnings[] }`

attached to the scenario (persisted read-only in `ai_metadata.scenarioIntegrity`). No stage boundary moves; the Scenario Builder still owns the data. `generationAllowed` is **always** `true`.

Milestones (ship small — measure after each):
- **1.5a** — Report scaffold + persona / expected-result / coverage-polarity checks (the three highest-signal, purely-textual checks).
- **1.5b** — Test-data suitability + missing-preconditions + step-completeness checks.
- **1.5c** — Business-flow consistency (deterministic state-order) + grounding-completeness (confidence penalty).
- **1.5d** — Automation Readiness Score aggregation + Scenario Quality dimension surfaced to the Quality Gate & dashboard (read-only, never blocking).

> **Sequencing:** only after the Scenario Integrity Validator is stable do we begin Sprint 2.

Sprint 2 — Candidate Resolution ⭐⭐⭐⭐⭐ (highest ROI)

> **Renamed from "Locator Ranking Engine," and now absorbs the former "Framework Reuse" sprint.**
> Locator selection is only one answer this sprint produces. The real question is broader:
>
> **"What is the best automation representation of this business action?"**

That representation might be a locator — but often the best answer is **no new locator at all**, because the framework already has a reusable abstraction. Candidate Resolution ranks all of these against each other:

- an existing **fixture** (e.g. `authenticatedFixture()`)
- an existing **page-object method** (e.g. `LoginPage.login()`)
- an existing **helper**
- an existing **component**
- ...and only then a **new locator**

```
Scenario:  "Click Login"

Candidate Resolution discovers:
   ✓ Existing LoginPage.login()          ← reuse wins
   ✓ Existing authenticatedFixture()     ← even better: skip the UI entirely
   ○ Need a new locator                  ← last resort
```

Sometimes the best locator is **no locator** — reuse of an existing abstraction is more valuable and more senior than any freshly-generated `page.fill(); page.fill(); page.click()`. This is where LevelUp beats almost everyone: **reuse, not AI.** (This is also why the old standalone "Framework Reuse" sprint no longer exists — reuse-vs-generate is the same decision as candidate resolution, and it must happen before any generation begins.)

Today

```
Grounding → Selector
```

Target

```
Grounding → Candidate Discovery → Candidate Ranking → Candidate Selection → Confidence
           (fixtures, page objects, helpers, components, locators — all candidates)
```

4.1 Candidate Discovery — collect, do not select

Gather candidates from every source, in priority order. **Do not choose yet.** Reusable framework assets are discovered first because reuse beats generation:

1. Existing **fixture** (highest reuse — may remove the need for UI steps entirely)
2. Existing **Page Object** method
3. Existing **helper** / **component**
4. **App Profile** (crawled grounding)
5. **Test Case** wording
6. **Expected Result** text
7. **Requirement** text
8. **DOM** relationships
9. **Accessibility** (role + name, aria-label, label association)
10. **AI** — last resort only

> **Fallback search chain before AI.** When the App Profile misses an element, do **not** jump to the LLM. Search deterministically first:
> `Test Case → Keywords → Expected Result → Requirement → Page Object → DOM → AI`
> Example — manual step `Click Login`, expected `Dashboard appears`: the composer infers Login causes navigation and searches for `login / sign in / submit / authenticate` candidates instead of hallucinating a selector.

4.2 Candidate Ranking — score every candidate

Reuse/extend `core/candidate-ranker.ts` scoring. Indicative scores:

Source / strategy	Score
Existing fixture (removes UI steps)	100
Existing Page Object method	100
Existing helper / component	99
`data-testid`	98
`role` + accessible name	96
`aria-label`	95
label association	94
placeholder	88
stable CSS	82
DOM fingerprint	78
XPath	40

Never just use the first locator. Find → Rank → Choose.

4.3 Candidate Selection — reuse first, then score the strategy, and record why

- **Existing Code First:** if a fixture / page-object / helper / component candidate resolves the action, prefer it over any new locator — even a perfect one. Reuse beats generation.
- Otherwise choose the highest-scoring candidate.
- If the top two are close, keep **both** — record the runner-up as an alternative.
- Score the **strategy** (reuse vs. locator, and which locator strategy), not just the string — and persist the **reasons**. Debuggability and auditability come for free: when someone asks “why didn’t LevelUp use XPath?” or “why did it call `login()` instead of filling the form?”, the answer is already stored.

```
Chosen:      LoginPage.login(user, pass)              strategy: reuse   score: 100
  Why:         ✓ existing page object  ✓ zero new code  ✓ matches framework convention
  Alternative: getByRole('button', { name: 'Login' })   strategy: role    score: 96 (reuse preferred)
```

4.4 Confidence banding (feeds the Confidence Engine sprint)

Internally scored `0–100`; **only the band is ever surfaced** (Hard rule #4):

Confidence	Behavior
High	Generate normally.
Medium	Generate normally **and** record `Review recommended.`
Low	Generate a `TODO` with a reason (e.g. Dynamic element) and a suggested approach.

Milestones (ship small — measure after each):
- **2a** — Candidate Discovery: collect candidates from all sources — **fixtures / page objects / helpers / components first**, then locators — into a typed list (no behavior change to selection yet).
- **2b** — Ranking: score candidates via the shared ranker; log chosen vs. alternatives.
- **2c** — Reuse-first selection + alternative retention (prefer an existing abstraction over a new locator).
- **2d** — Confidence banding (High/Medium/Low) + TODO emission for `Low`.

Sprint 3 — Assertion Expansion ⭐⭐⭐⭐⭐

Automation should validate **more** than the manual test, because it can. Deterministic expansion inside `AssertionEngine` — **not** prompting.

Input `Verify login successful` → **Output**

```
☑ URL changed
☑ Dashboard visible
☑ Logout button visible
☑ Session established
☑ No error present
☑ Inventory / landing content loaded
```

Rules: verify the **business outcome first**, then add one or two **stability** assertions. Never generate redundant assertions.

Sprint 4 — Coverage Expansion ⭐⭐⭐⭐⭐

Different from Sprint 3. Not more assertions — **more business coverage.**

Manual `Add item to cart` → **Automation additionally verifies**

- cart badge updated
- item count
- total price
- persistence (survives reload / navigation)
- navigation correctness

Meaningful, business-relevant checks only — never random assertions. Automation coverage should **always exceed** the manual test case.

> **Note — “Framework Reuse” is no longer a separate sprint.** Reuse-vs-generate is the same decision as candidate resolution, so it now lives inside **Sprint 2 — Candidate Resolution** and its “Existing Code First” heuristic (§5.1). Consulting `project-convention-profile` (`buildReuseCatalogue` , `findReusablePageObject`) happens there, before any generation begins:
>
> `login() ; login() ; login()  →  authenticatedFixture()`
>
> If `login()` (or any page-object method / fixture / helper) already exists, **reuse it — never regenerate.** The less code we generate, the more senior it looks.

Sprint 5 — Humanization ⭐⭐⭐⭐⭐

The framework must not feel generated. The problem is rarely correctness — it is that the output is **too symmetrical.** Senior engineers extract methods, simplify, and drop noise.

Reduce, inside `ai-review-engine` / `framework-auditor` :

- repetition
- unnecessary comments
- unnecessary variables
- over-defensive code
- identical naming everywhere

Make it feel like a teammate wrote it — following project conventions.

Sprint 6 — Confidence Engine ⭐⭐⭐⭐⭐

Last. Do not rewrite. Do not regenerate. Simply identify uncertainty and be honest about it, building on `engines/confidence-engine.ts` and the Sprint 2 bands.

Before returning, the composer asks: **“Would I merge this PR?”** — checking locator, assertion, reuse, compile, and framework confidence. When confidence is low, **flag**, don’t guess. Report the band (High/Medium/Low), never a raw number (Hard rule #4).

Smart TODOs (not AI noise)

Emit TODOs **only** when confidence is genuinely low — never a wall of `TODO TODO TODO` .

```
// Review locator:
// Dynamic element detected after login.
// Suggested locator:
// page.getByRole('button', { name: /checkout/i })
```

---

5. The two deterministic rule libraries (the secret sauce)

The system will have **exactly two** deterministic rule libraries — and no third intelligence layer. They complement each other without adding an AI component or growing prompts:

Library	Question it answers	Stage
Scenario Integrity Validator (§4, Sprint 1.5)	“Is the canonical scenario internally correct and automation-ready?”	Before the Composer
Engineering Heuristics Library (below)	“Given a correct scenario, what is the best way to implement it as production-quality automation?”	Inside the Composer

Together they give a stronger long-term foundation than continually expanding prompts or adding new AI components: one certifies the **input**, the other governs the **implementation** — both deterministic, both token-free, both versioned and unit-tested.

5.1 Engineering Heuristics Library

Not another intelligence. A **deterministic rule library** that captures how experienced automation engineers think. Every generation consults it. **No LLM. No prompt. No tokens.** It improves the product over time without growing prompt size.

Proposed home: `src/script-gen/heuristics/` (a pure, dependency-light rule module consumed by `ScriptGenEngine`). Rules are data + pure functions — unit-tested and versioned.

Rule families

Existing Code First ⭐ (the most valuable heuristic)

Before generating any new code, every generation must walk this chain and stop at the first hit:

```
Can an existing fixture solve it?
        ↓ no
Can an existing page object solve it?
        ↓ no
Can an existing helper solve it?
        ↓ no
Can an existing component solve it?
        ↓ no
Generate new code
```

This one rule does more than any other to eliminate duplication and make a generated framework feel like it was maintained by a real engineering team. It is the deterministic backbone of Sprint 2 (Candidate Resolution): reuse is always preferred over generation, even over a perfect new locator.

Locator heuristics (only when Existing Code First reaches “generate new”)
- Prefer existing page-object methods.
- Prefer `data-testid`.
- Prefer accessible roles and names.
- Avoid brittle CSS / XPath unless necessary.

Assertion heuristics
- Verify business outcome first.
- Add one or two stability assertions.
- Don’t generate redundant assertions.

Framework heuristics
- Reuse existing fixtures.
- Reuse page-object methods.
- Don’t create duplicate utilities.

Naming heuristics
- Follow the project’s existing naming conventions.
- Avoid identical/templated names everywhere.

Playwright heuristics
- No `waitForTimeout`; use web-first assertions and auto-waiting.
- Prefer user-facing locators (`getByRole` / `getByLabel`).

Logging heuristics
- Minimize logging statements — logs waste tokens that could improve assertions, waits, or locators.
- Default: log only test start/end and critical decision points, not every step.
- Every log must justify its existence vs. adding an assertion or refining a locator.

These heuristics evolve over time (the ongoing “continuous heuristic tuning” track improves the weights from successful generations) — **without** increasing token usage or architectural complexity.

---

6. Long-term stable roadmap

Priority	Goal	Change type	Why
1	Fix & Freeze	Regression fix + freeze declaration	Stabilize the foundation
1.5	**Scenario Integrity Validator**	**Certify the scenario (band + warnings); never mutate, never block**	The ONE validator
🥇 2	**Candidate Resolution** (absorbs Framework Reuse)	Reuse-first resolution of the best automation representation	**Reuse before generation**
🥈 3	Assertion Expansion	Increase automation coverage	Biggest quality gain users notice
🥉 4	Coverage Expansion	Add meaningful automation-only validations	Automation should exceed manual
5	Humanization	Output indistinguishable from hand-written code	Make code look hand-written
6	Confidence Engine	Flag uncertain areas instead of guessing	Flag uncertainty honestly
(ongoing)	Continuous heuristic tuning	Improve heuristic weights from successful generations	Better decisions, same architecture

Notice what is missing: no new orchestrators, no new intelligence providers, no new pipelines, no major rewrites — and **no new validators.** “Framework Reuse” is folded into Candidate Resolution because reuse-vs-generate is the same decision. Everything after the Validator is craftsmanship inside the Script Composer.

The three questions that gate every future sprint

The architecture is now strong enough. From here the mindset shifts from **“building architecture”** to **“building craftsmanship.”** Judge every proposed sprint against just three questions — if it improves none of them, do not build it, however interesting it is:

1. **Did the ready-to-run rate improve?** (less manual editing after generation)
2. **Did code reuse increase?** (more existing framework assets used, less duplication)
3. **Would an experienced Playwright reviewer be more likely to approve this PR without suspecting AI?**

7. PR checklist (paste into every Script Composer PR)

- [ ] Change lives **inside** the Script Composer / its owned sub-engines — no new orchestrator/provider/writer/planner/pipeline, no stage boundary moved.
- [ ] Prefers a **deterministic heuristic** over an LLM call where a decision can be made.
- [ ] Reuses existing framework assets (page objects, fixtures, helpers) before generating.
- [ ] Adds **no** unnecessary comments / logging / variables / defensive code.
- [ ] Preserves existing capabilities (no regressions); tests updated.
- [ ] Emits honest confidence / Smart TODOs instead of pretending certainty.
- [ ] Answers **yes** to: “Would a senior Playwright engineer merge this without realizing it was AI-generated?”
- [ ] References this document (`SCRIPT_COMPOSER_EVOLUTION.md`).

8. Appendix — component map (as of v1.0)

```
src/engines/scenario-builder.ts            canonical scenario (Phase A)
src/engines/scenario-integrity/ (proposed) Scenario Integrity Validator → Sprint 1.5 /
§5
src/renderers/scenario-renderer.ts         Manual / Script / BDD projection (Phase B)
src/script-gen/script-gen-engine.ts        ScriptGenEngine — the Script Composer
  ├── selector-quality-engine.ts           locator scoring         → Sprint 2
  ├── assertion-engine.ts                  assertions              → Sprints 3 & 4
  ├── wait-strategy-engine.ts              smart waits
  ├── workflow-mapper.ts                   navigation graph & flows
  ├── validation-runner.ts                 compile / validate       (Quality Gate)
  ├── ai-review-engine.ts                  style review             → Sprint 5
  └── framework-auditor.ts                 framework audit          → Sprint 5
src/intelligence/element-intelligence.ts   rankLocatorCandidates   → Sprint 2
src/intelligence/project-convention-profile.ts  reuse catalogue    → Sprint 2 (Candid-
ate Resolution)
src/core/candidate-ranker.ts               scored ranker (reuse)   → Sprint 2
src/core/advisors/*                        ranking advisors (reuse) → Sprint 2
src/engines/confidence-engine.ts           confidence              → Sprint 6
src/script-gen/heuristics/  (proposed)     Engineering Heuristics Library → §5
```